Mooc 1

TỔNG HỢP KIẾN THỨC TOÀN DIỆN: GENERATIVE AI IN SOFTWARE DEVELOPMENT

Tài liệu này là cẩm nang đúc kết toàn bộ kiến thức cốt lõi từ khóa học "Generative AI in software development", giúp bạn nắm vững lý thuyết, ứng dụng thực tiễn và các thách thức khi đưa AI vào doanh nghiệp.

PHẦN 1: TỔNG QUAN VỀ TRÍ TUỆ NHÂN TẠO & GENERATIVE AI

1. Các khái niệm nền tảng

- **Trí tuệ nhân tạo (AI):** Khái niệm rộng về việc máy móc mô phỏng trí thông minh con người.
- **Machine Learning (ML):** Là một tập con (subset) của AI. Cho phép máy tính tự học hỏi và cải thiện từ dữ liệu mà không cần lập trình các quy tắc cứng (if-else).
- **Trí tuệ nhân tạo tạo sinh (Generative AI - GenAI):** Là một nhánh của AI tập trung vào việc **tạo ra nội dung mới** (code, văn bản, âm nhạc, công thức nấu ăn...) từ các quy luật (patterns) đã học được từ lượng dữ liệu khổng lồ.

2. Cách GenAI hoạt động & Tương tác

- **Cơ chế:** Nhận diện các quy luật/mô hình trong tập dữ liệu lớn (Pattern recognition từ extensive training data), sau đó tạo ra kết quả dựa trên các quy luật đó.
- **Prompt Engineering (Kỹ nghệ câu lệnh):** Kỹ năng thiết kế câu lệnh đầu vào để tối ưu hóa phản hồi của GenAI.
    - *Nguyên tắc:* Cung cấp ngữ cảnh chi tiết (detailed context) cho các tác vụ phức tạp.
    - *Mẹo xử lý:* Nếu AI giải thích quá phức tạp, hãy yêu cầu một lời giải thích đơn giản hơn cho người mới bắt đầu (request a simpler explanation) hoặc yêu cầu cung cấp ví dụ minh họa (ask for examples).

PHẦN 2: AI TRONG PHÁT TRIỂN PHẦN MỀM (CODING & REFACTORING)

Các công cụ GenAI (Amazon Q, GitHub Copilot) không thay thế lập trình viên, mà hoạt động như một "trợ lý lập trình cặp" (Pair-programmer).

1. Các công cụ tiêu biểu

- **Amazon Q Developer:** Hỗ trợ gợi ý, hoàn thiện và gỡ lỗi (debug) code trực tiếp trong IDE. Có khả năng phân tích cấu trúc dự án để tự động tạo ra các mẫu code (templates) cho Controller, Service, Repository.
- **GitHub Copilot:** Nổi bật với tính năng tự động tạo gợi ý code và hoàn thiện các đoạn code

đang viết dở.
- **ChatGPT:** Rất mạnh trong việc giải thích các khái niệm lập trình phức tạp (VD: các concept khó trong Java) thành ngôn ngữ dễ hiểu.

2. Tái cấu trúc mã nguồn (Refactoring) & Code Review

- **Mục tiêu:** Giảm bớt các tác vụ lặp đi lặp lại (boilerplate code), nâng cao hiệu suất (performance) và khả năng đọc hiểu (readability) của mã nguồn.
- **Khả năng của AI trong Code Review:**
  - Kiểm tra code tự động (automated code inspection).
  - Nhận diện các đoạn code thừa thãi hoặc kém hiệu quả (inefficient/redundant sections).
  - Gợi ý module hóa (modularization) cho các hàm quá lớn.
  - Ép tuân thủ các tiêu chuẩn lập trình (enforcing coding best practices).
- **Ví dụ tối ưu hóa Java bằng AI:**
  - *Loại bỏ đồng bộ thừa:* Tránh việc sử dụng nhiều khối synchronized(this) liên tiếp trên cùng một đối tượng để giảm chi phí tài nguyên (overhead).
  - *Tối ưu Switch-case:* Gợi ý thay thế các câu lệnh switch dài dòng bằng Enum và gọi phương thức (method call).
- **Cảnh báo (Rủi ro):** AI có thể gợi ý các thiết kế không tuân theo các best practices hiện hành, do đó **luôn cần sự giám sát của con người**.

PHẦN 3: CÁC MÔ HÌNH RA QUYẾT ĐỊNH CỦA AI (AI DECISION-MAKING MODELS)

1. Cây quyết định (Decision Trees)

- **Đặc điểm:** Hoạt động như một sơ đồ tư duy. Dễ giải thích và trực quan (Easy to interpret and visualize).
- **Cấu trúc:** Các "nút bên trong" (internal nodes) đại diện cho các thuộc tính/đặc điểm (features) của dữ liệu.
- **Điểm yếu - Overfitting (Quá khớp):** Nếu cây có quá nhiều đặc điểm (features) và nhánh rẽ (splits), nó sẽ học vẹt dữ liệu cũ và dự đoán cực kỳ tệ trên dữ liệu mới.

2. Rừng ngẫu nhiên (Random Forests)

- **Khái niệm:** Là sự kết hợp của nhiều Cây quyết định lại với nhau để đưa ra dự đoán cuối cùng.
- **Kỹ thuật:** Sử dụng kỹ thuật **Bagging** (huấn luyện nhiều cây trên các tập con dữ liệu khác nhau).
- **Ưu điểm:** Cải thiện độ chính xác và giảm thiểu triệt để hiện tượng Overfitting so với một cây quyết định đơn lẻ. (Lưu ý: Random Forest không bị thiên kiến bẩm sinh).

3. Mạng Nơ-ron (Neural Networks)

- **Cấu trúc 3 lớp:**

    - *Lớp đầu vào (Input Layer).*
    - *Các lớp ẩn (Hidden Layers):* Nơi kết hợp và trích xuất các quy luật tiềm ẩn từ đầu vào. (Lưu ý: Đầu ra KHÔNG BAO GIỜ nằm ở lớp ẩn).
    - *Lớp đầu ra (Output Layer).*
- **Cách thức học tập:** Mạng nơ-ron học bằng cách liên tục **điều chỉnh trọng số (weights) và độ lệch (biases)** trong quá trình huấn luyện. Trọng số được cập nhật dựa trên phản hồi từ tỷ lệ lỗi dự đoán.
- **Hàm kích hoạt (Activation Function):** Đóng vai trò như "người gác cổng", quyết định thông tin nào được truyền qua nơ-ron. (VD: Hàm *Sigmoid* luôn trả về giá trị từ 0 đến 1, rất lý tưởng cho các bài toán xác suất).
- **Đánh giá (Validation):** Phải kiểm tra mô hình trên tập dữ liệu "chưa từng thấy" (unseen data) để đảm bảo mô hình có tính tổng quát (generalize) tốt, không bị thiên kiến (bias) và chịu được nhiễu.

PHẦN 4: CHUYỂN ĐỔI SỐ (DIGITAL TRANSFORMATION)

- **Định nghĩa:** Là quá trình tích hợp công nghệ kỹ thuật số vào mọi khía cạnh của doanh nghiệp để cải thiện hiệu quả và mang lại giá trị mới.
- **Các lĩnh vực chính (Key Domains):** Bao gồm Công nghệ (Technology), Dữ liệu (Data), và Quản trị sự thay đổi (Change Management).
- **Vai trò trung tâm của Dữ liệu:** Không phải chỉ để lưu trữ, mà để cung cấp **thông tin chi tiết có thể hành động (actionable insights)** nhằm thúc đẩy việc ra quyết định chính xác (drive decision-making).
- **Sự kết hợp công nghệ:**
    - *AI:* Tự động hóa quy trình, phân tích hành vi khách hàng để đưa ra đề xuất cá nhân hóa, phân tích dự đoán (predictive analytics) để dự báo nhu cầu hàng hóa.
    - *Cloud Computing:* Cung cấp lưu trữ dữ liệu có thể mở rộng (scalable) và tăng cường cộng tác.
    - *Internet of Things (IoT):* Giám sát thời gian thực và tối ưu hóa hàng tồn kho thông qua cảm biến.

PHẦN 5: THÁCH THỨC DOANH NGHIỆP & ĐẠO ĐỨC AI

Việc ứng dụng AI mang lại giá trị lớn nhưng đi kèm với các trách nhiệm bảo mật và đạo đức khắt khe.

1. Thách thức Triển khai & Mở rộng

- **Tích hợp (Integration):** Khó khăn thường gặp nhất là đảm bảo luồng dữ liệu liền mạch giữa giải pháp AI mới và các hệ thống di sản (legacy systems) như phần mềm CRM cũ.
- **Mở rộng (Scaling):** Khi hệ thống lớn lên, doanh nghiệp phải cân bằng giữa hạ tầng tiên tiến, thuật toán, điện toán đám mây và đặc biệt phải giải quyết bài toán cốt tử: **Bảo mật và**

quyền riêng tư dữ liệu ở quy mô lớn (Data privacy and security at scale).

2. Explainable AI - Tính giải thích được (Rất quan trọng)

- **Vấn đề:** Các mô hình AI thường giống như "hộp đen". Trong các lĩnh vực nhạy cảm (Y tế, Tài chính, Cho vay), các bên liên quan cần hiểu *tại sao* AI lại đưa ra quyết định đó.
- **Giải pháp công cụ:** Sử dụng **SHAP** hoặc **LIME** để minh bạch hóa mô hình. (VD: SHAP giúp tính toán và giải thích mức độ đóng góp của từng yếu tố như điểm tín dụng, thu nhập vào quyết định từ chối khoản vay).

3. Đạo đức (Ethics) & Tuân thủ pháp luật (Compliance)

- **Tính công bằng (Fairness):** Rủi ro lớn nhất là AI phản ánh lại những thiên kiến (bias) có sẵn trong tập dữ liệu huấn luyện. Doanh nghiệp cần đảm bảo sự công bằng và tránh phân biệt đối xử trong quá trình ra quyết định.
- **Nội dung tạo sinh:** AI có thể tạo ra thông tin sai lệch, gây hại, hoặc vi phạm bản quyền (nếu nó bắt chước các tác phẩm có bản quyền quá giống).
- **Tuân thủ quy định:** AI phải tuân thủ các luật bảo vệ quyền riêng tư như **GDPR**. Nguyên tắc đạo đức cao nhất là phải đảm bảo người dùng cung cấp **"sự đồng ý có hiểu biết" (informed consent)** trước khi thu thập và sử dụng dữ liệu cá nhân của họ cho các mục đích như quảng cáo mục tiêu.

Mooc 2

BÍ KÍP MASTER: KỸ NGHỆ YÊU CẦU & MÔ HÌNH HÓA HỆ THỐNG (UML)

Tài liệu tổng hợp trọn bộ kiến thức khóa học "Modeling Software Systems Using UML" & "System Requirements Capture".

PHẦN 1: TƯ DUY PHÁT TRIỂN PHẦN MỀM (SOFTWARE ENGINEERING MINDSET)

Trước khi bắt tay vào code hay vẽ biểu đồ, bạn phải có tư duy của một kỹ sư hệ thống.

1. Góc nhìn Vĩ mô vs. Vi mô

- **Programming-in-the-large (Lập trình vĩ mô):** Nhìn hệ thống từ trên cao. Bao gồm: Hiểu yêu cầu người dùng, áp dụng quy trình phần mềm (software process), xây dựng mô hình (models), đưa ra các quyết định đánh đổi thiết kế (design trade-offs).
- **Programming-in-the-small (Lập trình vi mô):** Chi tiết bên trong code. Bao gồm: Viết thuật toán, **kiểm tra dữ liệu đầu vào (validating user input)**, cấu trúc dữ liệu.

2. Giá trị của Quy trình và Thiết kế

- **Quy trình phần mềm (Software Process):** Lợi ích lớn nhất là tạo ra các mốc công việc rõ ràng, giúp **dễ dàng chia nhỏ và phân bổ công việc (divide up the work)** cho team.
- **Mục tiêu Thiết kế (Quality Design Goals):** Một thiết kế tốt (chia module rõ ràng) giúp giảm bớt sự phức tạp trong việc **hiểu hệ thống (understanding the system)**.

3. Quản trị rủi ro (Risk Management)

Trong dự án, rủi ro là tất yếu. Các chiến lược đối phó:

- **Avoid (Tránh):** Đổi kế hoạch/yêu cầu để rủi ro không xảy ra.
- **Mitigate (Giảm thiểu):** Chuẩn bị sẵn test/phương án phòng hờ.
- **Confine (Giới hạn):** Nhốt rủi ro lại, không cho lan rộng.
- **Monitor (Theo dõi):** Liên tục quan sát xem nó có xảy ra không.
- 🚫 **SAI LẦM CHẾT NGƯỜI: Ignore (Phớt lờ)** - Coi như rủi ro không tồn tại. Tuyệt đối không làm điều này!

PHẦN 2: THU THẬP & PHÂN TÍCH YÊU CẦU (REQUIREMENTS CAPTURE)

"Làm đúng sản phẩm quan trọng hơn làm sản phẩm đúng".

1. Functional vs. Non-functional Requirements

- **Functional (Chức năng):** Hệ thống *LÀM CÁI GÌ* (Ví dụ: Đăng ký sinh viên, Đăng nhập, Thêm vào giỏ hàng).
- **Non-functional (Phi chức năng):** Đặt ra các **ràng buộc (constraints)** về việc hệ thống hoạt động *TỐT NHƯ THẾ NÀO*.
    - *Ví dụ:* "Hệ thống phải đăng ký sinh viên **trong vòng chưa tới 1 giây**" -> Đây là Non-functional (Performance constraint).

2. Đặc tả & Xác thực yêu cầu (SRS & Validation)

- **SRS (System Requirements Specification):** Tài liệu Đặc tả Yêu cầu Hệ thống. Là bản hợp đồng kỹ thuật chi tiết nhất.
- **Nguyên nhân thất bại dự án:** Đa phần do lỗi lấy yêu cầu (Thiếu sự tham gia của user, kỳ vọng ảo, yêu cầu đổi liên tục). Tuy nhiên, **Lack of resources (Thiếu tiền/người)** là lỗi của Quản lý dự án, không phải lỗi của khâu Lấy yêu cầu.
- **Requirements Validation (Xác thực yêu cầu):** Cách tốt nhất để review là **sử dụng một danh sách câu hỏi kiểm tra (checklist)** cho từng yêu cầu (để xem nó có rõ ràng, đo lường được, khả thi không).

PHẦN 3: MÔ HÌNH HÓA USE-CASE (USE-CASE MODELING)

Đây là cầu nối giữa ngôn ngữ đời thường của khách hàng và ngôn ngữ kỹ thuật của developer.

1. Bản chất của Use Case

- **Góc nhìn:** Nắm bắt **hành vi của hệ thống (system behaviour)** từ góc nhìn của **người dùng (user's point of view)**.
- **Nó LÀ gì:** Mô tả Tương tác giữa Actor và System, biểu diễn các tính năng (functional).
- **Nó KHÔNG PHẢI là:**
    - **Không phải Giao diện người dùng (User Interface - UI):** Tuyệt đối không vẽ nút bấm hay màn hình vào Use Case.
    - **Không phải Cấu trúc dữ liệu:** Data để phần Domain Model lo.

2. Các thành phần của một Use-case Actor

- **Actor (Tác nhân):** Là người, thiết bị phần cứng, hoặc một hệ thống bên ngoài tương tác với phần mềm của chúng ta.
- **Lưu ý cực quan trọng:** Actor nằm NGOÀI hệ thống. Vì vậy, **Actor KHÔNG THỂ được biểu diễn bằng một Domain Model Class** (Class nghiệp vụ bên trong hệ thống).
- Đường nối giữa Actor và Use Case chỉ dùng **Association** (Liên kết thông thường).

3. Đặc tả chi tiết Use Case (Use-case Specification)

Một Use Case hoàn chỉnh phải được viết ra thành văn bản, bao gồm:

1. **Participating Actors:** Ai tham gia?
2. **Preconditions (Điều kiện tiên quyết):** Trạng thái "luôn phải đúng" trước khi bắt đầu kịch bản. (VD: Phải đăng nhập trước).
3. **Flow of events (Luồng sự kiện):** Bước 1, bước 2, bước 3... Hệ thống và Actor tương tác ra sao. Hành vi của Use Case được định nghĩa chính bởi phần này.
4. **Postconditions (Điều kiện hậu quyết):** Xong việc thì trạng thái hệ thống ra sao?

- 🚫 **TUYỆT ĐỐI KHÔNG đưa vào: Developing tools (Công cụ phát triển)** như IDE, Git, ngôn ngữ lập trình.

4. Luồng sự kiện (Flows)

- **Basic Flow:** Luồng thành công, suôn sẻ (Happy path).
- **Alternative Flow:** Các luồng rẽ nhánh do lỗi (error/exceptional), lựa chọn khác (variant), hành động tùy chọn (optional).
    - *Lưu ý:* Những hành vi được **dùng chung (shared)** bởi nhiều Use Case khác nhau thì không viết vào Alternative Flow, mà tách ra thành Use Case riêng và dùng quan hệ `<<include>>`.

PHẦN 4: UML CƠ BẢN VÀ MÔ HÌNH LỚP (CLASS & DOMAIN MODELING)

Khi đã rõ "Làm cái gì" (Use Case), ta chuyển sang "Thiết kế như thế nào" (Class/Domain Model).

1. UML là gì?

UML (Unified Modeling Language) không chỉ là hộp và mũi tên. Nó buộc chúng ta phải **suy nghĩ về thế giới theo một cách nhất định** (Tư duy Hướng đối tượng - chia mọi thứ thành Object).

2. Phân biệt Classifier và Feature

- **Classifier (Bộ phân loại):** Các khối xây dựng lớn như Class, Interface, Actor, Association.
- **Feature (Đặc tính):** Những thứ nằm *bên trong* Classifier. Điển hình là **Attribute (Thuộc tính)** và **Operation (Phương thức)**.
    - *Nghĩa là: Operation KHÔNG PHẢI là một Classifier.*

3. Cấu trúc của Class và Interface

- **Class (Lớp):** Đại diện cho một nhóm đối tượng chia sẻ 4 thứ: **Attributes, Operations, Relationships, và Semantics (Ngữ nghĩa).**
- **Interface (Giao diện):** Chỉ định nghĩa các dịch vụ mà class cung cấp cho bên ngoài. Do

đó, Interface **CHỈ chứa Operations** (không có Attributes).

4. Thuộc tính (Attribute) vs. Phương thức (Operation)

- **Thuộc tính:** Có kiểu dữ liệu, tính đa bội, khả năng hiển thị...
- **Phương thức:** Có tên, tham số, kiểu trả về. Tổ hợp này gọi là **Signature (Chữ ký)**.
    - *Nghĩa là: Signature KHÔNG PHẢI là tính chất của Attribute.*

5. Domain Modeling (Mô hình miền)

- Mục tiêu: Nắm bắt những thực thể quan trọng nhất của thế giới thực dưới dạng **Classes và Associations (Lớp và Mối liên kết)**.

PHẦN 5: CÁC MỐI QUAN HỆ TRONG UML (RELATIONSHIPS)

1. Association (Liên kết)

- Là mối quan hệ cấu trúc giữa 2 class.
- Đặc điểm: Luôn có thể **đọc và hiểu có ý nghĩa từ CẢ HAI chiều** (Bi-directional).
- Giữa 2 class bất kỳ, **LUÔN LUÔN CÓ THỂ (Yes, always)** có nhiều hơn 1 mối quan hệ Association (VD: Quan hệ "Là nhân viên" và quan hệ "Là quản lý").

2. Association Class (Lớp liên kết)

- **Khi nào dùng?** Khi bản thân **mối quan hệ SỞ HỮU CÁC THUỘC TÍNH (relationship has properties)** riêng của nó.
- *Ví dụ:* Class `KháchHàng` và `PhòngKháchSạn`. Quan hệ là "Đặt phòng". Thuộc tính "Ngày check-in" phải nằm ở Association Class `ĐơnĐặtPhòng`.

3. Generalization / Inheritance (Tổng quát hóa / Kế thừa)

Mối quan hệ Cha - Con (VD: Động vật -> Chó, Mèo). Có 4 Ràng buộc độ phủ (Coverage Constraints) cực kỳ quan trọng:

1. **Disjoint (Rời rạc):** Một đối tượng của class cha thuộc về **tối đa MỘT** class con.
2. **Overlapping (Chồng chéo):** Một đối tượng của class cha có thể thuộc về **NHIỀU** class con cùng lúc.
3. **Complete (Đầy đủ): Mọi** đối tượng của class cha **phải** thuộc về ít nhất một class con.
4. **Incomplete (Không đầy đủ): Có một số** đối tượng của class cha **KHÔNG** thuộc về bất kỳ class con nào.

Mooc 3

BÍ KÍP TỔNG ÔN KỸ THUẬT PHẦN MỀM (SOFTWARE ENGINEERING)

Tài liệu tổng hợp dành cho Dũng - Chinh phục khóa học Software Development

PHẦN 1: QUẢN LÝ DỰ ÁN & QUY TRÌNH PHÁT TRIỂN (SDLC)

1. Các khái niệm Quản lý cốt lõi

- **The Four P's (Mô hình 4 chữ P):** Để quản lý một dự án thành công, cần kiểm soát 4 yếu tố: **P**eople (Con người), **P**roduct (Sản phẩm), **P**rocess (Quy trình), và **P**roject (Dự án). *(Lưu ý: "Plan" không nằm trong 4P).*
- **Bước đầu tiên khi lập kế hoạch (Project Plan):** Luôn là **Define the scope** (Xác định phạm vi dự án). Không biết làm cái gì thì không thể lên lịch trình hay dự toán rủi ro.
- **Milestone (Cột mốc):** Là **A management decision point** (Điểm quyết định của ban quản lý). Tới mốc này, sếp sẽ xem xét có duyệt chi tiền để làm tiếp hay dừng lại.
- **Độ phức tạp của phần mềm (Complexity):** Thường đến từ quy mô (kích thước chương trình), độ khó của nghiệp vụ, hoặc thiếu hụt tài nguyên phần cứng (insufficient hardware). *Từ vựng chuyên ngành (vocabulary) ít gây ra sự phức tạp cho bản thân kiến trúc phần mềm nhất.*
- **Custom Software (Phần mềm tùy chỉnh):** Làm riêng cho 1 khách hàng. Đặc điểm cốt lõi là **Development effort is high** (Tốn rất nhiều công sức xây dựng từ đầu).

2. Các Mô Hình Phát Triển Phổ Biến

- **Waterfall (Thác nước):** Làm tuần tự từng bước. Ưu điểm lớn nhất là **Disciplined** (Tính kỷ luật, cấu trúc rất cao). Nhược điểm là khó sửa yêu cầu.
- **Agile (Linh hoạt):** Nhấn mạnh vào **Working software** (Phần mềm chạy được), con người, sự cộng tác và phản hồi thay đổi. *(Agile không ưu tiên các tài liệu hay kế hoạch test quá đồ sộ - Comprehensive test plan).*
- **Spiral (Xoắn ốc):** Đặc trưng lớn nhất là được dẫn dắt bởi **Risk Analysis** (Phân tích rủi ro) ở đầu mỗi vòng lặp.
- **Phân biệt Incremental và Iterative:**
    - **Incremental (Tăng trưởng/Cộng dồn):** Xây từng phần hoàn chỉnh. Từng đợt release sẽ có **Partial system, Full functionality** (Hệ thống một phần, nhưng tính năng đầy đủ. *Ví dụ: Xây xong hẳn phòng khách rồi mới xây phòng ngủ*).
    - **Iterative (Lặp vòng):** Xây phác thảo toàn bộ trước rồi làm mịn dần. Từng đợt release sẽ có **Full system, Partial functionality** (Toàn bộ hệ thống, nhưng tính năng chưa đầy đủ. *Ví dụ: Đổ móng và dựng khung cho cả cái nhà, rồi mới đi sơn tường sau*).

PHẦN 2: THIẾT KẾ KIẾN TRÚC & MÔ HÌNH HÓA (ARCHITECTURE & UML)

1. Thiết Kế Kiến Trúc (Architectural Design)

- **Định nghĩa:** Là quá trình xác định các **hệ thống con (sub-systems)** tạo nên toàn bộ hệ thống.
- **Mô hình Kiến trúc (Architectural Model):** Dùng để **thảo luận (discuss)** và **làm tài liệu (document)**. Nó chỉ hiển thị các khối chính, **không hiển thị các thành phần con (sub-components)** để tránh sự rắc rối.
- **3 Lợi ích vàng của Kiến trúc rõ ràng:**
    - Stakeholder communication (Giao tiếp với các bên liên quan).
    - System analysis (Phân tích hệ thống).
    - Large-scale reuse (Tái sử dụng quy mô lớn).
    - *(Lưu ý: "Performance evaluation" - Đo lường hiệu năng chi tiết KHÔNG phải lợi ích chính ở bước này).*

2. UML (Unified Modeling Language)

- **Generalization (Tổng quát hóa/Kế thừa):** Cho phép lớp con **Specialize** (chuyên biệt hóa) các thuộc tính và phương thức từ lớp cha (không được xóa bỏ chúng).
- **Multiplicity (Bản số):** Quy định **số lượng (number of objects)** bắt buộc/có thể tham gia vào một mối quan hệ.
- **{XOR} Constraint:** Nếu một Class có 2 đường nối XOR ra ngoài, đối tượng của nó chỉ được phép liên kết với **1 trong 2** (only one of two classes) tại một thời điểm.

PHẦN 3: TRIỂN KHAI & LẬP TRÌNH (IMPLEMENTATION)

- **Tại sao phải vẽ Mô hình (Model)?** Để giúp chúng ta **xử lý sự phức tạp** (deal with complexity) của vấn đề trước khi code.
- **Abstraction (Trừu tượng hóa):** Giấu chi tiết, chỉ hiện giao diện. Giúp **Understanding the system** (Hiểu hệ thống dễ hơn).
- **Encapsulation (Đóng gói):** Giới hạn quyền truy cập. Giúp **Maintaining the system** (Bảo trì/Sửa code dễ hơn mà không làm hỏng chỗ khác).
- **Refactoring (Tái cấu trúc code):** Viết lại code cho gọn gàng, dễ đọc để **Cải thiện cấu trúc bên trong (Improve internal structure)**, tuyệt đối không làm thay đổi tính năng hay sửa bug (sửa bug là Debug).
- **Defensive Programming (Lập trình phòng thủ):** Luôn viết code bắt lỗi để **Không cho phép dữ liệu đầu vào làm sập chương trình (do not let input crash your program)**.
- **Configuration Management (Quản lý cấu hình):** Bao gồm Git, Version, Release, System Building. *(Không bao gồm Risk management - Quản lý rủi ro).*

PHẦN 4: KIỂM THỬ PHẦN MỀM (TESTING)

Đây là phần nặng nhất, hãy chú ý kỹ các định lý sau:

1. Triết lý Kiểm thử

- **Mục đích của Testing:** Là để chứng minh phần mềm **CÓ LỖI (has defects)**, chứ không phải chứng minh nó hoàn hảo.
- **Sự thật về độ bao phủ:** Chạy test qua 100% các dòng code (Testing every statement) **KHÔNG BAO GIỜ** đảm bảo tìm ra mọi bugs.
- **Chi phí sửa lỗi:** Càng phát hiện lỗi muộn (gần lúc release), chi phí sửa lỗi **càng tăng mạnh (increases)**. Do đó, thiết kế test sớm (Early test design) tuy tốn công sức ban đầu nhưng giúp ngăn lỗi sinh sôi.
- **Static Testing:** Đánh giá tài liệu (Reviews/Inspections) mà không cần chạy code cũng được coi là Testing, vì nó giúp tìm lỗi và tăng chất lượng.

2. Hai Khái Niệm Dễ Nhầm Lẫn Nhất

- **Verification (Xác minh):** Trả lời câu "Làm phần mềm đúng kỹ thuật chưa?" (Functions work correctly, no defects). Đảm bảo code đúng như bản Thiết kế.
- **Validation (Thẩm định):** Trả lời câu "Làm đúng cái khách cần chưa?" (Meets the stated requirements). Đảm bảo giải quyết đúng bài toán thực tế.

3. Kỹ thuật Hộp Trắng (White-Box Testing)

- *Đặc điểm:* Bắt buộc **phải có Source Code** để xem bên trong.
- **Basis Path Testing:** Đảm bảo test mọi đường dẫn, qua đó chạy qua **Tất cả câu lệnh (all the statements)** ít nhất 1 lần.
- **Data Flow Testing:** Nhắm vào vòng đời và sự chính xác của **Các biến trong chương trình (program variables)**.
- **Loop Testing (Kiểm thử vòng lặp):**
    - **Vòng lặp lồng nhau (Nested):** Test vòng trong cùng trước, giữ vòng ngoài ở **giá trị nhỏ nhất (minimum value)**.
    - **Vòng lặp đơn (Simple):** Bắt buộc cần ít nhất **7 test cases** (0, 1, 2, m, n-1, n, n+1).

4. Kỹ thuật Hộp Đen (Black-Box Testing) & Khác

- *Đặc điểm:* Không cần Source Code, thường dùng trong **Integration Testing** hoặc System Testing.
- **Boundary Value Analysis (Giá trị biên):** Tester luôn test các giá trị ranh giới (ví dụ: pass 5.0 thì test 4.9, 5.0, 5.1) vì **lỗi rất hay xảy ra ở đây (errors are more likely to occur here)**.
- **Non-functional Testing (Kiểm thử phi chức năng):** Test các thuộc tính hệ thống như **Performance (Hiệu năng)**, Bảo mật, Tốc độ tải... chứ không test tính năng nghiệp vụ.

Mooc 4

CẨM NANG MASTER: PHÂN TÍCH, THIẾT KẾ VÀ ĐẢM BẢO CHẤT LƯỢNG HỆ THỐNG PHẦN MỀM

Tài liệu này tổng hợp các kiến thức tinh hoa nhất từ khóa học, được chia thành 6 mảng chính: Tổng quan Hệ thống, Mô hình hóa UML, Kiến trúc & Design Pattern, Đảm bảo chất lượng (SQA), Kiểm thử (Testing) và Quản lý dự án.

PHẦN 1: TỔNG QUAN VỀ PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Sự khác biệt cốt lõi: Phân tích là tìm hiểu xem hệ thống cần làm GÌ (What), còn Thiết kế là xác định LÀM NHƯ THẾ NÀO (How).

- **Mục đích của Phân tích và Thiết kế:** Một trong những mục đích chính là **điều chỉnh/áp dụng các yêu cầu vào môi trường triển khai thực tế** (*adapt the requirements to the implementation environment*).
- **Cách ly môi trường:** Khi thiết kế, ta cần đóng gói các yếu tố thuộc về môi trường triển khai (như database, framework) để **cách ly chúng khỏi các lớp ứng dụng chính** (*encapsulating it to isolate from application classes*). Điều này giúp lõi hệ thống không bị ảnh hưởng khi thay đổi công nghệ bên ngoài.
- **Giảm thiểu độ phức tạp:** Cách hiệu quả nhất để giảm độ phức tạp khi thiết kế một hệ thống phần mềm là phải **có các mục tiêu thiết kế rõ ràng ngay từ đầu** (*having clear design goals*). Mục tiêu rõ ràng giúp tránh tình trạng "Over-engineering" (thiết kế thừa thãi).

PHẦN 2: KIẾN TRÚC PHẦN MỀM & MÔ HÌNH HÓA (UML)

1. Kiến trúc MVC (Model - View - Controller)

Đây là kiến trúc kinh điển để phân tách các mối quan tâm (Separation of Concerns).

- **Model:** Đại diện cho **logic nghiệp vụ của ứng dụng** (*the business logic of the application*) và dữ liệu.
- **View:** Giao diện hiển thị cho người dùng.
- **Controller:** Xử lý các tương tác đầu vào từ người dùng và điều phối Model/View.

2. Mô hình phân tích ECB (Entity - Control - Boundary)

Hệ thống phân bổ các chức năng (use case) vào 3 loại class này nhằm mục đích **cô lập các loại thay đổi cụ thể vào các loại lớp cụ thể** (*isolate specific types of changes to specific types*

of classes), giúp hệ thống dễ bảo trì.

- **Entity Class (Lớp Thực thể):** Đại diện cho **các thông tin có tính bền bỉ, lưu trữ lâu dài** (*information that is persistent*). Ví dụ: Data khách hàng, Hóa đơn.
- **Boundary Class (Lớp Giao diện/Biên):** Nơi giao tiếp với người dùng hoặc hệ thống khác (UI, API).
- **Control Class (Lớp Điều khiển):** Chứa logic phối hợp, luồng công việc.
- **Nguyên tắc VÀNG (Best Practice): KHÔNG ĐƯỢC PHÉP có sự tương tác trực tiếp giữa Boundary Object và Entity Object.** Mọi giao tiếp phải đi qua Control Object.

3. Biểu đồ Trạng thái (State Machine Diagram)

- **Mục đích:** Mô tả **hành vi của MỘT ĐỐI TƯỢNG (an object)** trong suốt vòng đời của nó.
- **Trạng thái (State):** * Có thời lượng (duration).
    - Có thể được định hình bởi giá trị các thuộc tính hoặc liên kết với đối tượng khác.
    - **Luôn phải có tên** (KHÔNG thể anonymous/unnamed).
    - Có thể tự động chuyển trạng thái khi hoàn thành tác vụ nội bộ, không nhất thiết lúc nào cũng phải đợi sự kiện bên ngoài.
- **Sự kiện (Event) & Chuyển đổi (Transition):**
    - Biểu đồ chỉ phản hồi lại các sự kiện đóng vai trò **kích hoạt một sự chuyển đổi** (*triggers a transition*).
    - Sự kiện có thể là **sự trôi qua của một khoảng thời gian nhất định** (*passage of a designated period of time* - ví dụ: Timeout sau 10s).

PHẦN 3: CÁC MẪU THIẾT KẾ (DESIGN PATTERNS) & NGUYÊN TẮC

Design Patterns là các giải pháp ý tưởng cho các vấn đề lặp đi lặp lại. Chúng ĐƯỢC MÔ TẢ BẰNG VĂN BẢN/UML, KHÔNG PHẢI MÔ TẢ BẰNG MÃ NGUỒN CỤ THỂ (Not described using program code).

- **Tính gắn kết (Cohesion):** Một class được thiết kế tốt (gắn kết cao) khi nó **có các trách nhiệm liên quan chặt chẽ với nhau** (*has responsibilities that are closely related*).
- **Singleton Pattern:** Ngăn chặn việc tạo **nhiều hơn một phiên bản (instance)** của một class/biến.
- **Observer Pattern:** Dùng để **thông báo cho các đối tượng khác về những cập nhật** (*notify other objects about updates*) khi Subject thay đổi trạng thái.
- **Strategy Pattern:** Dùng để **đóng gói một họ các thuật toán có thể thay thế cho nhau** (*encapsulate an interchangeable family of algorithms*), cho phép thay đổi thuật toán ở runtime.
- **Bridge Pattern:** Dùng để **tách biệt một sự trừu tượng hóa khỏi việc cài đặt chi tiết của nó** (*decouple an abstraction from its implementation*), cho phép cả 2 phát triển độc lập.
- **Mediator Pattern:** Dùng để **đóng gói cách một tập hợp các đối tượng tương tác với nhau** (*encapsulate how a set of objects interact*), giảm thiểu sự phụ thuộc chéo (tight

coupling) giữa chúng.

PHẦN 4: ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM (SQA)

SQA là một bộ các hoạt động quản lý, quy trình (Process) để ngăn ngừa lỗi, khác với QC (Kiểm soát chất lượng - Product Testing) là tìm lỗi.

- **Các hoạt động của SQA:** Bao gồm thực thi tiêu chuẩn (standards enforcement), đánh giá tài liệu (artifact reviews), quản lý cấu hình (configuration management), và bao trùm cả kiểm thử. Tuy nhiên, **Đào tạo nhân sự (People training)** KHÔNG phải là một hoạt động thuộc SQA (nó thuộc về Quản lý nhân lực).
- **Chỉ số Cyclomatic Complexity:** Độ phức tạp này cho chúng ta biết về **tính an toàn - khả năng ít gặp lỗi** (*safety - how likely it is to be error free*). Code càng phức tạp, nguy cơ lỗi càng cao.
- **Quality of Conformance (Chất lượng tuân thủ):** Là mức độ mà quá trình sản xuất/lập trình bám sát đúng với các đặc tả thiết kế ban đầu.
- **Sổ tay tiêu chuẩn (Standards Handbook):** Là khung quy tắc của tổ chức. Dự án có thể sửa đổi, tạo mới, hoặc thỉnh thoảng bỏ qua một tiêu chuẩn có lý do, nhưng **KHÔNG THỂ tự ý quyết định không tuân theo bất kỳ tiêu chuẩn nào**.
- **Mô hình CMM & PCMM:** Mục đích của SEI-CMM (quy trình) và PCMM (con người) là để **đánh giá và cải thiện các thực tiễn hiện tại** (*assess and improve current practices*) của một tổ chức phần mềm.

PHẦN 5: KIỂM THỬ PHẦN MỀM (SOFTWARE TESTING)

- **Behavioral Testing (Kiểm thử hành vi):** Chính là **Black box testing (Kiểm thử hộp đen)** - chỉ quan tâm đầu vào/đầu ra, không nhìn vào code.
- **State-based Testing (Kiểm thử theo trạng thái):** Các Test case cho phương pháp này được lấy ra/dẫn xuất từ chính các **State machine diagrams** (Biểu đồ trạng thái).
- **Kỹ thuật phân tích giá trị biên (Boundary Value):** Để tìm số test case tối thiểu (ví dụ: test ngày trong tuần). Nếu có 7 ngày hợp lệ, bạn cần tối thiểu 7 test case hợp lệ + ít nhất 1 test case không hợp lệ = 8 test values.
- **Các cấp độ kiểm thử (Levels of Testing):**
    - **Unit Testing (Kiểm thử Đơn vị):** Test từng class/module riêng lẻ.
    - **Integration Testing (Kiểm thử Tích hợp):** Tập trung vào **sự tương tác giữa các components/modules** (*the interaction among the components*).
    - **System Testing (Kiểm thử Hệ thống):** Kiểm tra toàn bộ hệ thống đã tích hợp, thường do **đội ngũ phát triển/QA thực hiện** (*done by the developers*).
    - **Acceptance Testing (Kiểm thử Chấp nhận - UAT):** Do **khách hàng/Client thực hiện** ở giai đoạn cuối. Mục đích của kế hoạch UAT là **xác định các tiêu chí xem hệ thống đã thực sự hoàn thành chưa** (*specify the criteria for determining whether the system is finished*).

PHẦN 6: QUẢN LÝ DỰ ÁN (PROJECT MANAGEMENT)

- **Biểu đồ Gantt (Gantt Chart):** Là công cụ xuất sắc để quản lý tiến độ, thời lượng, tài nguyên dự án. Tuy nhiên, bản thân Biểu đồ Gantt cơ bản **KHÔNG THỂ hiện trực quan được các công việc nằm trên Đường Găng - Critical Path** (*the tasks that lie on the critical path*). Để làm điều này một cách chính xác, người ta phải dùng các sơ đồ mạng lưới như PERT/CPM.